

Stop Dreaming, Start Engineering

Cloud Patterns for
Production AI

Jeremy Meiss — Tech Solution Architect

//

How many of you have shipped an AI demo that quietly died?

80%+

of AI projects fail - twice the rate of conventional IT

RAND, 2024. 65 practitioner interviews - citing an outside estimate.

Where the failure actually is

The notebook

`model.fit()` converges

Accuracy looks great on the holdout set

Demo day: it works

The system around it

Data pipeline - unowned, undocumented

Identity boundary - nobody drew it

Observability - bolted on after the incident

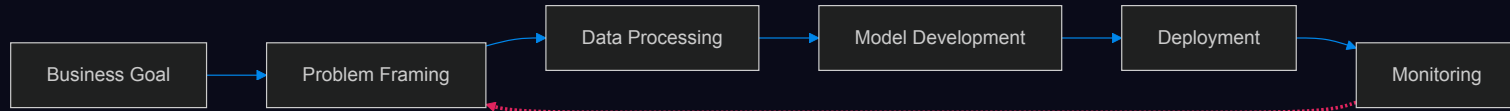
Cost line - nobody modeled it



//

AI is software with new rules.

The six-phase lifecycle



Well-Architected, for AI

Operational Excellence

MLOps automation

Security

Model, data, prompt

Reliability

Recovery from drift

Performance

Purpose-built silicon

Cost

Inference is your bill

Sustainability

Measurable now

Three competitors, the same six answers - arrived at independently. That's not marketing. That's emerging engineering truth.

AWS ML Lens · GCP AI/ML Framework · Azure Well-Architected AI workload guidance



Well-Architected, for AI



1 / 28



Shipping non-
deterministic code

The core problem

A unit test

Same input →
one deterministic output
Pass or fail

The same test, against an LLM

Same input →
three different outputs, three runs
"Correct" isn't pass or fail

Four patterns, one slide

01

Blue- Green

Fast rollback, trust
your validation

02

Canary

Gradual, real-user
exposure

03

Shadow

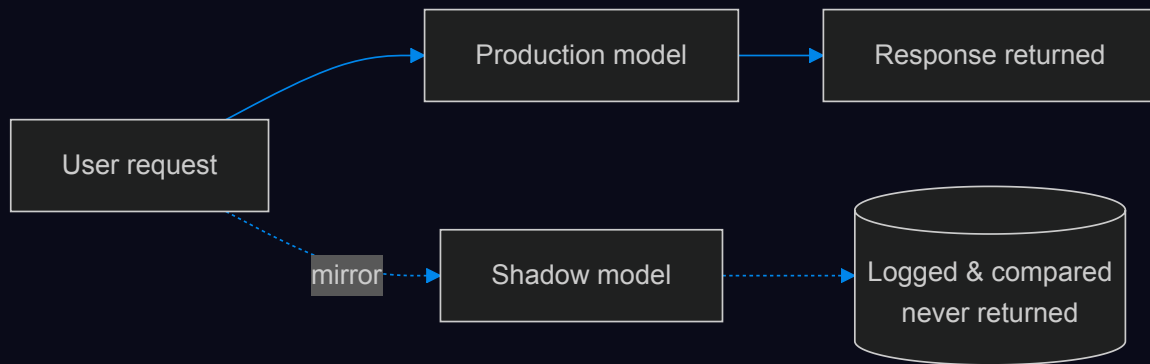
Mirror traffic,
discard results

04

A/B Testing

Prove business
impact

Shadow deployment, in detail



Build 1 - production path only

Build 2 - shadow duplication added

Build 3 - comparison and diff, labeled

The Azure template

01

**Deploy
green**

0% traffic

02

**Smoke
test**

Invoke
green by
name

03

Mirror

Live traffic
duplicated

04

**Progressive
shift**

10% → 25% → 50%
→ 100%

05

**Retire
blue**



The sequence

01

Shadow

Tells you whether the model
is broken

02

Canary

Tells you whether users
tolerate it

03

Full cutover

//

If shadow isn't in your pipeline, that's your highest-leverage fix.



The protocol that
made agents
portable

What is an agent, actually

Model + tool

Input → arrow → output

Not an agent

Model in a loop

Tools, observations feeding back

Agent

New failure modes

Infinite loops

Wrong tool called

Hallucinated tool

Correct tool, wrong argument

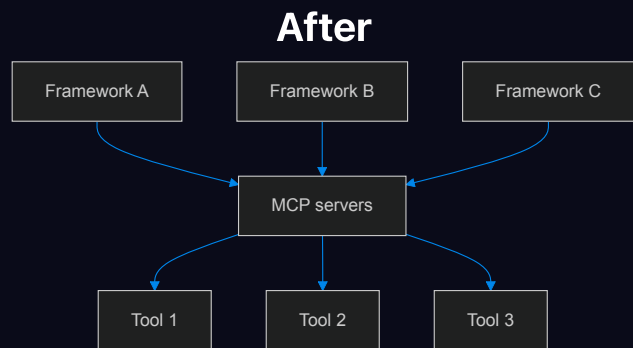
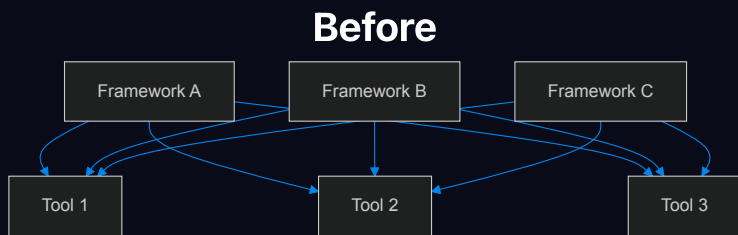
Correct result, misinterpreted

None of these existed in request/response.

Your orchestration pattern decides which of these you fight.



MCP: the setup



Model Context Protocol. Anthropic, November 2024. Open standard.

Six orchestration patterns

Sequential

Structured pipelines

Routing

Clear category dispatch

Parallel

Latency reduction,
diverse perspectives

ReAct

Tool use, iterative

Hierarchical (Magnetic)

Open-ended tasks

Evaluator

Quality-critical outputs

Magnetic-One: Microsoft Research



Runtime landscape, current state

AWS AgentCore

GA Oct 2025. Policy, Evaluations, and managed harness all GA June 17, 2026 (AWS Summit NY). Payments (Coinbase, Stripe) announced.

GCP Agent Platform

Announced Cloud Next '26. Vertex retired. Skill Registry. Memory Bank profiles GA.

Microsoft Foundry

Agent Framework 1.0 GA. Foundry Agent Service GA. Hosted Agents targeted GA early July 2026. M365 publishing.

AWS Summit New York, June 2026 · Google Cloud Next '26 · Microsoft Build 2026



//

In 2026, agents run everywhere. What varies is who they answer to.



The part that
determines your
platform choice

The two identity patterns

Identity passthrough

Microsoft Foundry + Fabric

User's Entra token flows through the agent

Agent inherits the human

Runtime isolation

GCP Agent Identity

SPIFFE cert bound to container, mTLS everywhere

Agent has its own identity, tied to where it runs



OpenTelemetry GenAI conventions

```
gen_ai.system: "anthropic"  
gen_ai.request.model: "claude-opus-5"  
gen_ai.usage.input_tokens: 1834  
gen_ai.usage.output_tokens: 412  
gen_ai.tool.name: "search_knowledge_base"  
gen_ai.server.time_to_first_token: 0.312
```

Datadog · Honeycomb · Grafana · LangChain · CrewAI · AutoGen

The de facto standard. Not yet formally Stable.

As of July 2026 - OpenTelemetry GenAI semantic conventions, Development status

//

Platform choice matters more than model choice.

The nine questions

1. Are your feedback loops wired from monitoring back to framing?
2. Do you have a shadow stage before real users?
3. Classic or agentic retrieval - deliberate choice?
4. Which orchestration pattern? Can you draw it in 30 seconds?
5. Are your tools exposed via MCP?
6. Can your agent ever see data its user can't? Structurally, not policy?
7. Are you emitting OpenTelemetry GenAI conventions?
8. Article 50 enforceable since 08 Aug 2026 - are you ready?
9. Do you know your per-inference cost and have a routing strategy?

Thank You

Jeremy Meiss

 jerdog.dev

 /in/jeremy-meiss

 jerdog

 jmeiss.me

Slides: <https://github.com/jerdog/talk-cloud-ai-engineering-patterns>

Thank You

Jeremy Meiss

 [jerdog.dev](https://twitter.com/jerdog)

 [/in/jeremy-meiss](https://www.linkedin.com/in/jeremy-meiss)

 [jerdog](https://github.com/jerdog)

 jmeiss.me

Slides: <https://github.com/jerdog/talk-cloud-ai-engineering-patterns>